# Fault Detection and Isolation - Report

Emmeran Colot

## I. Introduction

This report covers the different implementations of fault detection and isolation methods for discrete-time systems. It shows and analyses the results obtained based on the theoretical explanations given by Prof. M. Kinnaert in the *Model-Based and Data-Driven Fault Detection and Isolation* course.

The discrete-time state space model is given by

$$\begin{cases} x[k+1] = Ax[k] + Bu[k] + E_d d[k] + E_f f[k] \\ y[k] = Cx[k] + Du[k] + G_d d[k] + G_f f[k] \end{cases} \tag{1}$$

where $x \in \mathbb{R}^{n\times 1}$ is the state, $u \in \mathbb{R}^{m\times 1}$ the input to the system, $y \in \mathbb{R}^{p\times 1}$ the output of the system, $d \in \mathbb{R}^{n_d\times 1}$ the disturbance and $f \in \mathbb{R}^{n_f\times 1}$ the fault. The exact values for those matrices are given in Appendix A.

## II. Parity relations based method

### A. Overview of the method

Based on the state space representation (1), $y[k]$ can be expressed as a function of the state at time instance $k-s$ and the input, disturbance and fault from time instance $k-s$ up to $k$. This was derived by recurrence and the resulting equation, in a matrix form, is

$$Y_s(k) = \Theta_s x(k-s) + T_s^u U_s(k) + T_s^d D_s(k) + T_s^f F_s(k). \tag{2}$$

The "*signal matrices*" are built as follows

$$Y_s(k) = \begin{bmatrix} y[k-s] \\ \vdots \\ y[k] \end{bmatrix} \quad \cdots \quad F_s(k) = \begin{bmatrix} f[k-s] \\ \vdots \\ f[k] \end{bmatrix}, \tag{3}$$

and the "*weight matrices*":

$$\Theta_s = \begin{bmatrix} C \\ CA \\ \vdots \\ CA^s \end{bmatrix}, \quad T_s^u = \begin{bmatrix} D & \mathbf{0} & \cdots & \mathbf{0} \\ CB & D & \ddots & \vdots \\ \vdots & \ddots & \ddots & \mathbf{0} \\ CA^{s-1}B & \cdots & CB & D \end{bmatrix}$$
$$T_s^{d/f} = \begin{bmatrix} G_{d/f} & \mathbf{0} & \cdots & \mathbf{0} \\ CE_{d/f} & G_{d/f} & \ddots & \vdots \\ \vdots & \ddots & \ddots & \mathbf{0} \\ CA^{s-1}E_{d/f} & \cdots & CE_{d/f} & G_{d/f} \end{bmatrix} \tag{4}$$

Because of their nature, disturbances are unknown (stochastic noise, nonlinearities, unmeasured input signal, ...) and the state is often not measured. To only keep from (2) the fault and the measured signals, they are split from the other terms and both sides of the equation is then multiplied by $\Omega_s$.

The unknown side of the equation can then be cancelled with a clever choice for $\Omega_s$, i.e. a basis of the left null space of $\begin{bmatrix}\Theta_s & T_s^d\end{bmatrix}$:

$$\Omega_s \left(Y_s(k) - T_s^u U_s(k) - T_s^f F_s(k)\right) = \mathbf{0}. \tag{5}$$

It is worth mentioning the existence condition for $\Omega_s$, which depends on the rank of the matrix to cancel:

$$\text{rank}\begin{bmatrix}\Theta_s & T_s^d\end{bmatrix} < (s+1)p \tag{6}$$

This leads to the definition of the parity vector, which is equal to $\mathbf{0}$ in case there is no fault and is computed by

$$r[k] = \Omega_s \left(Y_s(k) - T_s^u U_s(k)\right). \tag{7}$$

### B. Fault detection

Applied to a system with 1 input and 2 outputs, a simulation has been done with $s = 2$. It contains a sinusoidal disturbance and a fault on the sensors. Fig. 1 shows what is computed by the simulation in a case where no fault is present.

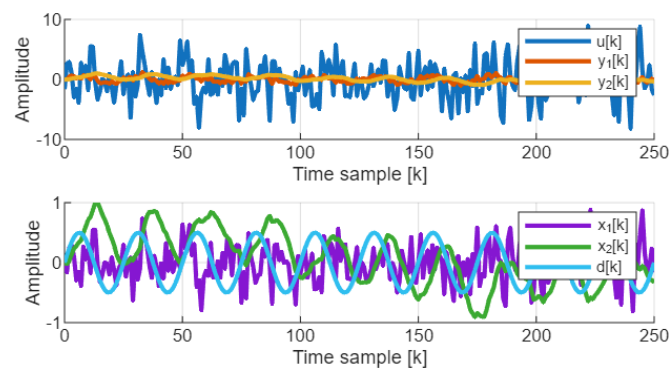


Fig. 1. Measured signals (top) and internal signals (bottom) in a faultless scenario.

The simulation is ran with a constant bias error on each of the two sensors and the resulting parity vector norm is shown on Fig. 2.

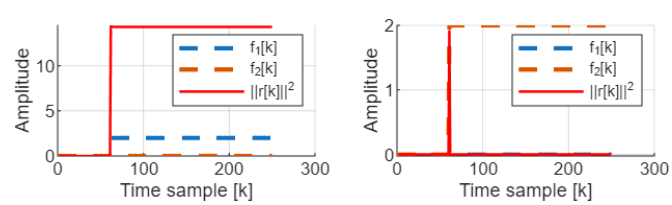


Fig. 2. Applied fault (on state 1 on the left plot and state 2 on the right plot) and resulting residual.

The residual on the right subplot indicates that the fault can be detected but is forgotten after a few samples. This

is because the second state integrates the input, meaning a constant error will not be detected. On the other hand, the first state is simply amplifying the input. A constant difference between the expected output and the measured one is therefore easily detected.

For the same reason, a ramp-shaped fault will generate a ramp shaped-residual when applied on the first state and a constant residual in the other case, as shown on Fig. 3.

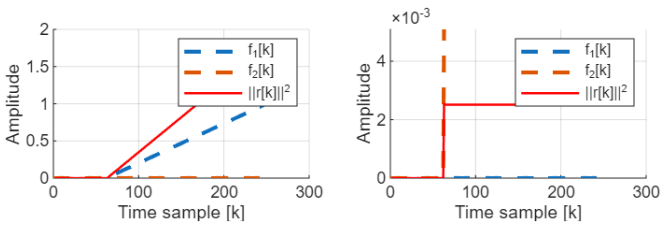


Fig. 3. Applied fault (on state 1 on the left plot and state 2 on the right plot) and resulting residual. The second fault looks like a step due to the scale difference between the parity vector norm and the fault signal.

### C. Fault Isolation

To isolate the fault, i.e. determine which is the faulty sensor, residuals must be crafted by considering some faults as disturbances using a confusion matrix. Because on the existence condition on the left null space of $\begin{bmatrix}\Theta_s & T_s^d\end{bmatrix}$ given by (6), a residual generally cannot be built to control a single fault.

To understand how fault isolation works, the system is slightly modified by adding a second sensor on the first output. Three distinct residuals are then built, each targeting all but one fault according to the following confusion matrix:

| | $f_1$ | $f_2$ | $f_3$ |
|---|---|---|---|
| $r_1$ | 0 | $\times$ | $\times$ |
| $r_2$ | $\times$ | 0 | $\times$ |
| $r_3$ | $\times$ | $\times$ | 0 |

To generate $r_1$, the disturbance matrices $E_d$, $G_d$ and the fault matrices $E_f$, $G_f$ are modified. By moving some columns from the fault matrices to the disturbance matrices as follows,

$$\begin{cases} E_d = \begin{bmatrix} E_{d_1} \end{bmatrix} \\ E_f = \begin{bmatrix} E_{f_1} & E_{f_2} & E_{f_3} \end{bmatrix} \\ G_d = \begin{bmatrix} G_{d_1} \end{bmatrix} \\ G_f = \begin{bmatrix} G_{f_1} & G_{f_2} & G_{f_3} \end{bmatrix} \end{cases} \Rightarrow \begin{cases} E_d = \begin{bmatrix} E_{d_1} & E_{f_1} \end{bmatrix} \\ E_f = \begin{bmatrix} E_{f_2} & E_{f_3} \end{bmatrix} \\ G_d = \begin{bmatrix} G_{d_1} & G_{f_1} \end{bmatrix} \\ G_f = \begin{bmatrix} G_{f_2} & G_{f_3} \end{bmatrix} \end{cases} \tag{8}$$

the residual $r_1$ will now only detects faults on the 2$^{\text{nd}}$ and 3$^{\text{rd}}$ sensors. The same approach allows to generate 3 residuals, and therefore a fault can be isolated.

## Appendix A
## Exact Model

The model used in the simulations is, in continuous time,

$$\dot{x}(t) = A_c x(t) + B_c u(t) + E_{d,c} d(t) + E_{f,c} f(t), \tag{9}$$

$$y(t) = C_c x(t) + D_c u(t) + G_{d,c} d(t) + G_{f,c} f(t), \tag{10}$$

with

$$A_c = \begin{bmatrix} b\eta_1 & 0 \\ 1 & 0 \end{bmatrix}, \quad B_c = \begin{bmatrix} b \\ 0 \end{bmatrix}, \quad E_{d,c} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \tag{11}$$

$$E_{f,c} = \mathbf{0}, \quad \text{and} \quad \begin{cases} b = 2.2 \\ \eta_1 = -10 \end{cases}. \tag{12}$$

The output equation matrices are

$$C_c = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad D_c = 0, \tag{13}$$

$$G_{d,c} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad G_{f,c} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}. \tag{14}$$

The discrete-time model in (1) is finally obtained using a ZOH discretization with sampling time $T_s = 0.2$ seconds.